

JavaScript Avanzado (1.^a parte)

Resumen detallado — Frontend · Fastbook de Robert Da Corte (Qualentum Lab)

Este fastbook profundiza en los mecanismos internos de JavaScript y en las técnicas modernas para trabajar con asincronía. Está pensado para quienes ya dominan los fundamentos del lenguaje y quieren perfeccionar sus habilidades. A continuación se resumen los 8 bloques del temario, destacando los conceptos clave de cada uno.

1. Callbacks

Un callback es **una función que se pasa como argumento a otra función** y que se ejecuta después de que la función principal termine. Es la base histórica de la programación asíncrona en JavaScript, usada en temporizadores, lecturas de archivos (Node.js) o peticiones a servidores.

Punto clave: encadenar muchos callbacks anidados produce el conocido *callback hell* (o "pirámide de la perdición"), un código difícil de leer y mantener. La solución pasa por modularizar en funciones separadas o adoptar promesas y `async/await`.

2. Event loop

JavaScript es **monohilo (single-threaded)**: solo puede ejecutar una instrucción a la vez. Aun así, puede realizar tareas no bloqueantes (AJAX, temporizadores, etc.) gracias al event loop y a tres piezas clave:

- **Call stack (pila de llamadas):** almacena las funciones en ejecución, funcionando como una pila LIFO (la última función en entrar es la primera en salir). Si se acumulan demasiadas llamadas, se produce un *stack overflow*.
- **Callback queue (cola de llamadas):** almacena los callbacks de operaciones asíncronas ya completadas, esperando a ser ejecutados.
- **Web APIs:** funcionalidades proporcionadas por el entorno (navegador o Node.js) —no por el propio lenguaje— como DOM, Fetch API, `setTimeout`, `WebSocket`, etc. Permiten delegar tareas fuera del hilo principal.

Funcionamiento: el event loop vigila constantemente el call stack; en cuanto este queda vacío, mueve los callbacks pendientes de la callback queue al call stack, uno a uno y en orden de llegada, para que se ejecuten.

3. Closures (clausuras)

Una closure es una función que **recuerda el entorno (scope) en el que fue creada**, incluso después de que la función externa que la contenía haya terminado de ejecutarse. Esto permite que una función interna siga accediendo a variables de su función externa.

Utilidad práctica: las closures se usan para emular variables privadas: valores que no se pueden modificar directamente desde fuera, solo a través de funciones específicas.

4. Gestión de errores

La estructura **try / catch / finally** permite identificar y responder a errores en tiempo de ejecución:

- **try**: contiene el código que puede fallar.
- **catch (error)**: captura y gestiona el error si se produce; recibe un objeto *error* con las propiedades **error.name** (tipo de error) y **error.message** (descripción).
- **finally**: se ejecuta siempre, haya habido error o no (útil para ocultar spinners, cerrar conexiones, etc.).

JavaScript incluye varios tipos de error incorporados: **ReferenceError** (variable inexistente), **TypeError** (uso incorrecto de un valor), **SyntaxError**, **RangeError**, **EvalError** y **URIError**. También se pueden lanzar errores propios con la palabra clave **throw**.

5. Promesas y async/await

Una **promesa (Promise)** es un objeto que representa el resultado futuro de una operación asíncrona, ofreciendo un enfoque más estructurado que los callbacks. Tiene tres estados posibles:

- **Pending (pendiente)**: estado inicial, aún no resuelta ni rechazada.
- **Fulfilled (cumplida)**: la operación se completó con éxito.
- **Rejected (rechazada)**: la operación falló.

Se crea con **new Promise((resolve, reject) => {...})** y se consume con **.then()** (éxito), **.catch()** (error) y **.finally()** (siempre). Las promesas pueden encadenarse para ejecutar operaciones asíncronas en secuencia.

Métodos útiles del objeto Promise:

- **Promise.all()**: se cumple cuando todas las promesas del array se cumplen, o se rechaza si alguna falla.
- **Promise.race()**: se resuelve o rechaza tan pronto como lo hace la primera promesa del array.
- **Promise.any()**: se resuelve en cuanto una promesa se cumple; si todas fallan, rechaza con un **AggregateError**.
- **Promise.allSettled()**: espera a que todas las promesas terminen (cumplidas o rechazadas), sin importar el resultado.

async/await (ES6) permite escribir código asíncrono con apariencia síncrona, mejorando la legibilidad. Una función marcada como **async** siempre devuelve una promesa, y dentro de ella **await** pausa la ejecución hasta que la promesa se resuelve. Importante: **await** solo puede usarse dentro de funciones **async**.

6. This

El valor de **this** depende del contexto en que se invoca la función, no de dónde se define:

- **Contexto global**: al llamar una función normal, **this** apunta al objeto global (**window** en el navegador, **global** en Node.js).
- **Métodos de objeto**: **this** se refiere al objeto que contiene el método.

- **Constructores/clases:** con el operador `new`, `this` apunta al nuevo objeto creado.
- **Eventos:** en manejadores de eventos, `this` suele referirse al elemento que disparó el evento.
- **Arrow functions:** no tienen `this` propio; heredan el `this` del entorno donde fueron definidas.

`call()`, `apply()` y `bind()` permiten fijar o cambiar manualmente el contexto de `this`: **`call()`** pasa argumentos de forma individual, **`apply()`** los pasa como un array, y **`bind()`** devuelve una nueva función con el `this` ya vinculado permanentemente.

7. Modularización

Modularizar consiste en dividir el código en piezas reutilizables (módulos) para mantenerlo organizado, evitar colisiones de nombres y facilitar el mantenimiento. El fastbook repasa cuatro sistemas:

Sistema	Descripción	Uso típico
IIFE (Immediately Invoked Function Expression)	Función que se ejecuta inmediatamente tras definirse; encapsula variables para que no contaminen el ámbito global.	Técnica clásica para que no se contaminen el ámbito global.
ESM (ES Modules)	Sintaxis nativa de ES6 con las palabras clave export / import.	Estándar actual en navegadores y Node.js
CommonJS	Usa module.exports para exportar y require() para importar.	Node.js (predominante antes de ESM)
AMD (Asynchronous Module Definition)	Usa define() para declarar módulos y cargarlos de forma asíncrona.	Navegadores, vía RequireJS
UMD (Universal Module Definition)	Patrón que combina CommonJS y AMD, con fallback al ámbito global.	Bibliotecas que deben funcionar en cualquier entorno.

Aunque ESM es hoy el estándar dominante, conocer CommonJS, AMD y UMD sigue siendo relevante por la cantidad de código heredado y bibliotecas que aún los emplean.

8. Conclusiones del fastbook

El curso cierra reforzando las ideas centrales: los **callbacks** como base de la asincronía, el **event loop** (call stack + callback queue + Web APIs) como motor que ordena la ejecución, las **closures** para mantener estado vivo, la **gestión de errores** como práctica imprescindible, las **promesas y async/await** como evolución natural frente a los callbacks, el control del contexto con **this**, **call**, **apply** y **bind**, y la **modularización** (IIFE, ESM, CommonJS, AMD, UMD) como pilar de la organización del código.

Cuadro rápido de conceptos clave

Concepto	Idea central
Callback	Función pasada como argumento, ejecutada tras completarse otra
Event loop	Coordina call stack, callback queue y Web APIs para la asincronía
Closure	Función que recuerda el entorno donde fue creada
try/catch/finally	Estructura estándar para capturar y gestionar errores
Promise	Objeto que representa el resultado futuro (pending/fulfilled/rejected)
async/await	Sintaxis que hace legible el código asíncrono como si fuera síncrono
this	Referencia al objeto de ejecución; depende de cómo se invoca la función
call/apply/bind	Métodos para fijar o cambiar el contexto (this) de una función

Modularización

División del código en piezas reutilizables (IIFE, ESM, CommonJS, AMD, UMD)

Documento generado a partir de: JavaScript Avanzado (1.ª parte) — Robert Da Corte / Qualentum Lab.